# Quantum VRP Implementation Write-Up

July 23, 2025

**Abstract**

This document details the implementation of a Quantum Approximate Optimization Algorithm (QAOA) for solving a simplified Vehicle Routing Problem (VRP). It delves into the problem's quantum encoding, the specifics of the QAOA circuit (including the role of various quantum gates), and the classical optimization process. The write-up also covers the step-by-step progress, technical hurdles encountered during development, the achieved outputs, and addresses the critical issue of noise in quantum hardware, proposing future directions for this research.

## 1 Introduction

The Vehicle Routing Problem (VRP) is a fundamental combinatorial optimization challenge in logistics, aiming to determine the most efficient routes for a fleet of vehicles to service a set of customers. Given its NP-hard nature, classical algorithms often struggle with large-scale instances due to computational limitations. Quantum algorithms, particularly the Quantum Approximate Optimization Algorithm (QAOA), are emerging as promising candidates to tackle such problems on quantum computers. This write-up provides a practical account of implementing QAOA for a VRP instance, specifically in the context of waste management as outlined in the accompanying research proposal.

## 2 Problem Formulation: VRP and Quantum Encoding

The success of quantum optimization for VRP hinges on effectively translating the classical problem into a quantum language that can be processed by a quantum computer. This involves a clear encoding scheme and a well-defined cost function.

### 2.1 VRP Instance Details

The implementation utilizes VRP data from a JSON file (`cluster_complete_graphs.json`), focusing on a `waste_facility` with ID 5. The dataset provides a comprehensive distance matrix between various geographical nodes. Key parameters extracted for the current problem instance include:

- **Nodes:** [5, 22, 48, 57, 70, 87]
- **Depot:** 5 (This is the central waste facility)
- **Customers:** [22, 48, 57, 70, 87] (These are the waste collection areas to be visited)
- **Total Nodes (N):** 6 (Depot + Customers)
- **Number of Customers (n):** 5

### 2.2 Node-Visit-Time Encoding (Step 1)

The chosen encoding strategy is the "Node-Visit-Time Encoding", which represents the sequence of customer visits. Instead of directly encoding paths, this method assigns a discrete "time slot" or "order" to each customer node.

To represent these time slots using qubits, we determine the number of bits required for each node. If there are $N$ total nodes (depot + customers), then each customer's visit time can range from 0 to $N - 1$. The number of bits needed to uniquely identify these $N$ possible time slots is $\lceil \log_2(N) \rceil$.

For our instance:

- **Bits per node ($\lceil \log_2(6) \rceil$):** 3 bits are required to represent time slots from 0 to 5. For example, $000_2 = 0$, $001_2 = 1$, ..., $101_2 = 5$.
- **Total Qubits required:** Since there are 'num_customers' customers and each needs 'bits_per_node' bits to encode its visit time, the total number of qubits is Number of Customers $\times$ Bits per node $= 5 \times 3 = 15$ qubits.

A quantum state (bitstring) of these 15 qubits is interpreted by grouping the bits for each customer. For example, if we have customers A, B, C, D, E and each requires 3 bits, a 15-bit string 'b14 b13 ... b0' would be parsed as:

- Bits [2:0] for Customer A's visit time.
- Bits [5:3] for Customer B's visit time.
- ...and so on.

These decoded integer visit times are then used to sort the customers, forming the sequence of visits. The full route starts at the depot, visits customers in the determined order, and returns to the depot.

### 2.3 Cost Function

The objective of QAOA is to minimize a classical cost function that measures the "goodness" of a solution. For this VRP, the cost function for a decoded route encompasses two main parts:

1. **Total Distance Cost:** This is the sum of Euclidean (or road network) distances between consecutive nodes in the generated route, starting from the depot, visiting all customers, and returning to the depot. The distances are retrieved from the pre-calculated 'distance_matrix_km'.
2. **Penalty Costs:** These are added to penalize routes that violate VRP constraints, making them less favorable for the optimizer.
    - **Duplicate Visit Penalty (`PENALTY_DUPLICATE_VISIT` = `1000.0`):** This large penalty is applied if two or more customers are assigned the exact same visit time. In a valid VRP solution, each customer must be visited exactly once, implying unique visit times for distinct customers.
    - **Out-of-Range Penalty (`PENALTY_OUT_OF_RANGE` = `1000.0`):** This penalty applies if a decoded visit time for any customer falls outside the valid range of 0 to $N - 1$. This ensures that the quantum measurements yield meaningful time slot assignments.

The optimizer seeks to find quantum parameters that minimize this total cost, thereby favoring routes with minimal distance and no constraint violations.

## 3 Quantum Approximate Optimization Algorithm (QAOA) Implementation

QAOA is a hybrid quantum-classical algorithm. It uses a quantum circuit to prepare a state that encodes the solution to an optimization problem, and a classical optimizer to iteratively refine the parameters of this quantum circuit.

### 3.1 Conceptual Link to VRP Cost Function (Implicit Hamiltonian)

In a fully formalized QAOA, the VRP is typically first converted into a Quadratic Unconstrained Binary Optimization (QUBO) problem or an Ising Hamiltonian. This Hamiltonian, $H_C$, represents the problem's objective and constraints. For example:

$$H_C = \sum_{i<j} J_{ij} Z_i Z_j + \sum_i h_i Z_i$$

where $Z_i$ is the Pauli Z operator on qubit $i$, and $J_{ij}$ and $h_i$ are coefficients derived from the VRP distances and penalties.

The QAOA then applies a unitary operator $U_C(\gamma) = e^{-i\gamma H_C}$, which is approximated using a sequence of quantum gates corresponding to the terms in $H_C$. Similarly, a mixer Hamiltonian, often $H_M = \sum_i X_i$, is used to generate $U_M(\beta) = e^{-i\beta H_M}$, which encourages exploration of the solution space.

### 3.2 QAOA Circuit Construction (`create_qaoa_circuit`)

The current implementation uses a general QAOA circuit structure for 'p' layers (depth 'p=2' in this instance). The parameters $\gamma$ and $\beta$ are the angles for quantum gates.

1. **Initial Layer:** A Hadamard gate (H) is applied to each of the 'num_qubits'. This creates an initial uniform superposition state over all possible bitstrings, ensuring that all potential solutions are explored.

$$|\psi_0\rangle = \bigotimes_{i=0}^{N_q-1} \mathrm{H}\,|0\rangle = \frac{1}{\sqrt{2^{N_q}}} \sum_{x\in\{0,1\}^{N_q}} |x\rangle$$

2. **Cost Layer ($U_C(\gamma)$):** This layer implements the phase rotations corresponding to the problem's cost Hamiltonian. In this simplified implementation, we use:

   - **Single-qubit $R_Z$ gates:** $R_Z(2\gamma_k)$ applied to each qubit $i$. The $R_Z(\theta)$ gate implements a rotation around the Z-axis of the Bloch sphere by an angle $\theta$:

$$R_Z(\theta) = \exp\left(-i\frac{\theta}{2}Z\right) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

     These gates are typically used to encode single-qubit terms ($Z_i$) of the cost Hamiltonian.

   - **Two-qubit $R_{ZZ}$ gates:** $R_{ZZ}(2\gamma_k)$ applied to all pairs of qubits $(i, j)$. The $R_{ZZ}(\theta)$ gate implements a rotation around the ZZ-axis for two qubits:

$$R_{ZZ}(\theta) = \exp\left(-i\frac{\theta}{2}Z\otimes Z\right)$$

     These are used to encode two-qubit interaction terms ($Z_iZ_j$) of the cost Hamiltonian.

   The 'gamma_k' parameter (from the classical optimizer) scales the strength of these rotations. It's important to note that while these gates are part of a cost layer, the explicit coefficients from the VRP QUBO (distances, penalties) are *not* directly encoded into individual gate angles. Instead, the 'qaoa_objective_function' evaluates the complete classical cost of the measured bitstring, which allows for this flexible classical evaluation of cost, common in QAOA demonstrations for complex problems.

3. **Mixer Layer ($U_M(\beta)$):** This layer allows the quantum state to explore different configurations. It typically consists of Pauli-X rotations.

   - **Single-qubit $R_X$ gates:** $R_X(2\beta_k)$ applied to each qubit $i$. The $R_X(\theta)$ gate implements a rotation around the X-axis of the Bloch sphere by an angle $\theta$:

$$R_X(\theta) = \exp\left(-i\frac{\theta}{2}X\right) = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

     These gates represent the mixer Hamiltonian $H_M = \sum_i X_i$, which helps in tunneling between computational basis states.

   The 'beta_k' parameter (from the classical optimizer) scales these mixer rotations.

After the 'p' layers are applied, all qubits are measured in the computational basis. The resulting bitstrings are then used by the classical optimization loop.

```
def create_qaoa_circuit(num_qubits, p, gamma, beta):
    """
    Creates a QAOA circuit for a given depth p and parameters gamma, beta.
    This uses a simplified cost layer for demonstration.
    """
    qc = QuantumCircuit(num_qubits, num_qubits) # num_qubits for classical bits for
    measurement

    # Initial layer: Apply Hadamard to all qubits
    qc.h(range(num_qubits))
    qc.barrier()
```

```
    for k in range(p):
        # Cost Layer (U_C): Apply problem Hamiltonian (simplified)
        # For VRP, this would encode distances and constraints.
        # Here, we use RZ for single-qubit terms and RZZ for two-qubit interactions.
        # The actual cost is evaluated classically based on measurement outcomes.
        for i in range(num_qubits):
            qc.rz(2 * gamma[k], i) # Single qubit terms
        for i in range(num_qubits):
            for j in range(i + 1, num_qubits):
                # Apply RZZ gate for interactions between all pairs (can be optimized
    for specific problem)
                qc.rzz(2 * gamma[k], i, j)
        qc.barrier()

        # Mixer Layer (U_M): Apply mixer Hamiltonian
        # Typically a sum of Pauli X operators, implemented with RX gates.
        for i in range(num_qubits):
            qc.rx(2 * beta[k], i)
        qc.barrier()

    # Measure all qubits
    qc.measure(range(num_qubits), range(num_qubits))

    return qc
```

Listing 1: The `create_qaoa_circuit` function

### 3.3 Objective Function for Classical Optimization (`qaoa_objective_function`)

This function is the bridge between the quantum hardware/simulator and the classical optimizer. For a given set of parameters $(\gamma, \beta)$:

1. It constructs the QAOA circuit as described above.
2. It *transpiles* the circuit: this step optimizes the circuit for the specific target quantum backend's architecture (e.g., connectivity, native gates).
3. It submits the transpiled circuit to the 'Sampler' primitive. The 'Sampler' runs the circuit multiple times ('SHOTS=256') and returns the probability distribution of measurement outcomes (bitstring counts).
4. It calculates the **expected cost**: For each observed bitstring, it classically decodes the bitstring into a VRP route and calculates its cost using the 'total_cost_from_bitstring' function (which includes distance and penalties). The expected cost is then computed as the sum of (probability of bitstring * cost of bitstring).
5. This expected cost is the value that the classical optimizer seeks to minimize.

## 4 Implementation Steps and Progress

The development of the quantum VRP solver followed a structured, incremental approach.

### 4.1 Step 0: Loading VRP Data

The initial step involved loading the geographical and distance data for the VRP from 'cluster_complete_graphs.json'. This crucial foundation ensured that the VRP instance (depot, customers, distance matrix) was accurately represented for subsequent processing.

### 4.2 Step 1: Qubit Allocation

This step translated the classical VRP problem size into the required quantum resources. The Node-Visit-Time encoding determined the number of qubits necessary (15 for 5 customers and 6 total nodes), establishing the scale of the quantum circuit.

### 4.3 Step 2: Backend Selection

A connection to the IBM Quantum service was established, with 'ibm_brisbane' selected as the preferred quantum backend. This choice implies targeting a real quantum processor. In cases where 'ibm_brisbane' is unavailable or for faster local debugging, the code gracefully falls back to the 'AerSimulator'. A 'Sampler' primitive, essential for running quantum circuits and obtaining measurement probabilities, was then initialized.

### 4.4 Step 3: Classical Optimization of QAOA Parameters

This iterative phase is where the heart of the QAOA lies. The 'scipy.optimize.minimize' function, employing the 'COBYLA' optimization method, was used to search for the optimal $\gamma$ and $\beta$ parameters.

- **Initialization:** Random initial parameters were used to start the search.
- **Parameter Bounds:** Appropriate bounds were applied to the $\gamma$ and $\beta$ parameters to ensure they remained within physically meaningful ranges.
- **Iteration Control:** The 'maxiter=10' option was set to limit the optimization to 10 objective function evaluations, balancing solution quality with computational time.

Throughout this step, detailed print statements provided real-time feedback on the 'Current parameters', circuit processing (creation, transpilation), and the 'Sampler job completed in X.XX seconds' time, along with the 'Expected Cost'.

### 4.5 Step 4: Running QAOA with Optimal Parameters

Once the classical optimizer had converged (or reached 'maxiter'), the 'optimal_gamma' and 'optimal_beta' parameters were used to construct the final QAOA circuit. This circuit was then executed on the 'Sampler' to produce the final set of measurement outcomes.

### 4.6 Step 5: Processing Results and Interpreting Optimal Route

The final measurement counts were analyzed to identify the 'most_probable_bitstring'. This bitstring, representing the highest probability outcome from the quantum execution, was then decoded into a concrete VRP route (e.g., '5 -¿ 22 -¿ 48 -¿ 57 -¿ 70 -¿ 87 -¿ 5'). The total cost of this route, including distances and any penalties, was then calculated and presented.

## 5 Hurdles Encountered and Solutions

The development process was iterative, involving several debugging cycles to overcome specific technical challenges:

1. **Program Not Stopping (Initial Observation):**
   - **Hurdle:** The classical optimizer, without an explicit termination condition, could potentially run indefinitely, consuming computational resources.
   - **Solution:** The 'scipy.optimize.minimize' function was configured with 'options='maxiter': 10'. This critical addition provided a clear stopping point for the optimization process, making it manageable for testing and demonstration.
2. **Incorrect Sampler Result Access ('AttributeError: 'DataBin' object has no attribute 'meas'"):**
   - **Hurdle:** An 'AttributeError' occurred when attempting to retrieve raw measurement data from the 'Sampler' result using 'final_result[0].data.meas'. This indicated a misunderstanding of the Qiskit Runtime 'Sampler''s output structure.
   - **Solution:** Investigation revealed that classical bit measurements from the 'Sampler' are provided under the '.c' attribute of the 'DataBin' object. The line was corrected to 'final_meas_data = final_result[0].data.c', allowing successful data retrieval.

3. **Backend Name Access Error ('TypeError: 'str' object is not callable'):**
    - **Hurdle:** A 'TypeError' arose from attempting to call 'backend.name()' when checking the backend type for transpilation ('if backend.name() != 'aer_simulator':').
    - **Solution:** In Qiskit's backend objects, the 'name' is an attribute, not a method. The erroneous parentheses were removed, correcting the line to 'if backend.name != 'aer_simulator':'.
4. **COBYLA 'MAXFUN' Warning:**
    - **Hurdle:** A 'UserWarning' was encountered: 'COBYLA: Invalid MAXFUN; it should be at least num_vars + 2; it is set to 6'. This implied that the optimizer was internally capping its maximum function evaluations, despite a higher 'maxiter' being specified.
    - **Explanation:** For the 'COBYLA' algorithm, the minimum number of function evaluations ('MAXFUN') required for proper operation is 'num_variables + 2'. With 'QAOA_P=2', there are 4 parameters (2 $\gamma$, 2 $\beta$), leading to a minimum 'MAXFUN' of $4 + 2 = 6$. The warning signifies that while 'maxiter=10' was requested, COBYLA effectively capped its iterations to 6 for this problem size. Although it meets the minimum, the warning serves as a notice about this internal adjustment. For more exhaustive optimization, a significantly higher 'maxiter' (e.g., 50 or 100) would be specified, allowing COBYLA to run more iterations if needed, beyond its minimum requirement.

# 6 Problem with Noise in Quantum Cloud Hardware

When executing quantum circuits on real quantum hardware, such as 'ibm_brisbane', the presence of noise is a significant challenge. Current quantum processors (often referred to as Noisy Intermediate-Scale Quantum, or NISQ, devices) are susceptible to various forms of noise:

- **Decoherence:** Qubits lose their quantum properties (superposition and entanglement) over time due to interaction with their environment.
- **Gate Errors:** Imperfections in the control pulses used to implement quantum gates lead to slight deviations from the intended unitary operations. **Measurement Errors:** Errors can occur during the readout of qubit states, where a measured 0 might actually have been a 1, and vice-versa.
- **Crosstalk:** Operations on one qubit can unintentionally affect neighboring qubits.

These noise sources can dramatically impact the performance of quantum algorithms like QAOA. Noise accumulates with circuit depth and number of gates, leading to a degradation of the prepared quantum state. As a result, the measured bitstring probabilities deviate from their ideal (noiseless) distributions.

For QAOA, this means:

- The true optimal solution (bitstring) might have a lower probability of being measured.
- Suboptimal or even invalid solutions (those with high penalties) might receive artificially inflated probabilities.
- The "landscape" of the objective function (the expected cost) becomes noisy, making it harder for the classical optimizer to find the true minimum.
- The observed 'Most probable bitstring: ... (Count: 1, Probability: 0.004)' with a very low probability, even for a small number of shots and layers, is indicative of a highly mixed state, which can be exacerbated by noise on a real device. It suggests that no single bitstring strongly dominates the measurement outcomes.

Addressing noise is a critical area of research in quantum computing, and techniques like error mitigation are essential for obtaining meaningful results from NISQ devices.

# 7 Achieved Outputs

The current implementation successfully executes the end-to-end QAOA workflow for the specified VRP instance. A representative output illustrating the iterative optimization process (Step 3) is shown below:

```
STEP 3: Starting classical optimization for QAOA (p=2 layers)...
  [QAOA Obj] Current parameters: gamma=[0.13 0.4 ], beta=[3.82 0.32]
  [QAOA Obj] Creating QAOA circuit...
  [QAOA Obj] Circuit created. Transpiling...
  [QAOA Obj] Circuit transpiled. Running on Sampler...
  [QAOA Obj] Sampler job completed in 131.32 seconds. Processing results...
  [QAOA Obj] Calculating expected cost from measurements...
  [QAOA Obj] Expected Cost calculated: 2497.715
Expected Cost: 2497.715
  [QAOA Obj] Current parameters: gamma=[1.13 0.4 ], beta=[3.82 0.32]
  [QAOA Obj] Creating QAOA circuit...
  [QAOA Obj] Circuit created. Transpiling...
  [QAOA Obj] Circuit transpiled. Running on Sampler...
  [QAOA Obj] Sampler job completed in 6.85 seconds. Processing results...
  [QAOA Obj] Calculating expected cost from measurements...
  [QAOA Obj] Expected Cost calculated: 2584.254
Expected Cost: 2584.254
% ... (additional iterations demonstrating cost function evaluation) ...
```

Listing 2: Example Optimization Progress during Step 3

Upon completion of the optimization and final circuit execution, the script yields the following results:

```
Optimization finished.
Optimal Parameters: gamma=[0.133 0.398], beta=[4.822 1.321]
Minimum Expected Cost found: 2380.139

STEP 4: Running QAOA circuit with optimal parameters...

STEP 5: Processing results and interpreting optimal route...
Most probable bitstring: 010100000110100 (Count: 1, Probability: 0.004)
Decoded visit times for customers [22, 48, 57, 70, 87]: [0, 1, 2, 3, 4]
Optimal VRP Route: 5 -> 22 -> 48 -> 57 -> 70 -> 87 -> 5
Total Cost of Optimal Route (Distance + Penalties): 2380.139

Script finished successfully!
```

Listing 3: Example Final Output after Optimization

The low probability for the most probable bitstring (0.004) underscores the challenges with NISQ devices, limited shots, and a relatively shallow ‘p=2‘ QAOA circuit for a 15-qubit problem. It indicates that the quantum state, even after optimization, is still a superposition where no single solution dominates overwhelmingly, which is expected behavior for early-stage QAOA implementations on real hardware.

# 8 Next Steps

To further advance this quantum VRP implementation and address the challenges of current quantum hardware, the following avenues for future work are proposed:

1. **Enhanced QAOA Circuit Design and Hamiltonian Mapping:**
   - **Increasing ‘p‘ Layers:** Systematically investigate the impact of increasing the number of QAOA layers (‘QAOA_P‘). While this increases circuit depth, it allows for more complex state preparation and potentially better approximations of the optimal solution.
   - **Problem-Specific Hamiltonian Encoding:** Implement the VRP cost and constraints more rigorously as a QUBO/Ising Hamiltonian. This would involve deriving specific coefficients for $Z_i$ and $Z_iZ_j$ terms and mapping them directly to the angles of the corresponding $R_Z$ and $R_{ZZ}$ gates in the cost layer, moving beyond the current generic application.
2. **Advanced Classical Optimization Strategies:**
   - **Alternative Optimizers:** Evaluate other classical optimizers available in ‘scipy.optimize‘ (e.g., ‘SLSQP‘, ‘L-BFGS-B‘, ‘Nelder-Mead‘) or even more specialized optimizers designed for quantum variational algorithms (e.g., from Qiskit’s optimization module).

- **Parameter Initialization and Annealing Schedules:** Explore more intelligent ways to initialize the $\gamma$ and $\beta$ parameters, and investigate annealing-like schedules for their evolution during optimization, which can improve convergence.

3. **Robust Constraint Handling:**

   - **Capacity Constraints:** Integrate vehicle capacity limitations into the QUBO formulation and the cost function, making the solutions more practical.
   - **Time Windows:** Extend the model to account for specific time windows during which customer visits must occur. **Multiple Vehicles:** Adapt the encoding and cost function to accommodate routing for a fleet of multiple vehicles from the depot.

4. **Quantum Error Mitigation (QEM):**

   - Given the significant impact of noise on NISQ devices, implementing QEM techniques is crucial. This includes methods like Zero-Noise Extrapolation (ZNE), Readout Error Mitigation, and Dynamical Decoupling, to extract more accurate results from noisy measurements.

5. **Scalability and Performance Analysis:**

   - Conduct comprehensive studies on how the algorithm scales with increasing numbers of nodes and customers.
   - Benchmark the QAOA approach against classical VRP solvers (e.g., the naive brute-force solver provided in 'naive_vrp.py' for small instances, or more advanced heuristics/exact solvers for larger ones) in terms of solution quality, runtime, and resource utilization.

6. **Alternative Quantum Approaches:**

   - Explore the applicability of other quantum algorithms for VRP, such as Quantum Annealing (if access to annealers like D-Wave is available) or other variational algorithms like the Variational Quantum Eigensolver (VQE) for directly finding the ground state of the VRP Hamiltonian.

These next steps aim to enhance the current implementation's robustness, accuracy, and applicability to more complex and realistic VRP scenarios, moving closer to leveraging quantum computing for practical optimization challenges.